hydra_satsuma: Certified Structure Dispatch with a satsuma-iter+kissat Fall-Through

Greg Sidebottom

Research note — logic repo (https://github.com/gsidebottom/logic), SAT track, 2026-08-12; revised 2026-09-16 with the factoring stage (§1.2, §5.2) and 2026-10-04 with the circuit stage, the `hydra_circuit_satsuma` *arm (§1.5, §5.3) and October reruns of the satsuma-based arms under one set of conditions. Solver description for the* `hydra_satsuma` *backend of* `sat`*: the certified* `hydra` *structure-dispatch pipeline (Cook-shape prover, factoring stage, XOR/GF(2) stage and, in* `hydra_circuit_satsuma`*, a circuit stage) whose fall-through engine is satsuma-iter+kissat, winner of the SAT Competition 2026 main track. Includes the completed apples-to-apples comparison (*`hydra` */* `satsuma` */* `hydra_satsuma` */* `hydra_circuit_satsuma`*) on the official 2026 benchmark set, all on one Mac mini (M4 Pro, 64 GB; §2.1).*

Abstract

`hydra_satsuma` is a structure-dispatch SAT solver: before any CDCL search, it tests the input for algebraic structure that admits a *polynomial-size certificate*, and only when no structure applies does it hand the formula to a general-purpose engine. Four structural stages run in order: (1) a *Cook-shape prover* that recognizes five cardinality-contradiction families (pigeonhole and its generalizations) and emits the Cook-1976 extension-variable refutation as a VeriPB proof — families on which resolution, and hence any DRAT-based route without new variables, is exponential; (2) a *factoring stage* that recognizes a multiplier circuit whose product is pinned to a number N, reads N off the circuit by probing it, factors N numerically and rebuilds the model by propagation — the satisfiable instances of the ezfact and pyhala-braun families in milliseconds, the model as witness; a number with no fitting factor pair is recorded but never reported as a verdict, because primality has no short certificate in any accepted proof system; (3) an *XOR/GF(2) stage* that recovers parity constraints from their clausal encodings and runs Gaussian elimination, certifying parity refutations in VeriPB by Gocht–Nordström's division-based method (GN21); (4) in the `hydra_circuit_satsuma` variant, a *circuit stage* that reads the gates off the clauses and, where a multiplier's partial products lie hidden in nested multiplexers on the inputs, recovers them as extension variables and re-encodes the circuit from cuts of up to six leaves — each pass emitting a DRAT prefix that derives its output from its input, so that CaDiCaL's refutation of the rewritten formula, appended to the prefixes, is a `drat-trim`-checked refutation of the original. The fall-through engine is satsuma-iter+kissat (Anders et al.), the SAT Competition 2026 main-track winner, run unmodified in a container: satsuma iterates simplify–order–break rounds, adding only unit and binary breaking clauses, and emits a binary substitution-redundancy (SR) proof justifying each addition, kissat appends its clausal refutation to the same file, and `dsr-trim` verifies the *composed* proof against the exact CNF handed over — so symmetry-broken UNSAT results remain checker-certified. Every UNSAT path thus produces a machine-checked certificate in a format on the SAT Competition 2026 checker menu (VeriPB) or in the winner's own SR chain; SAT results carry the model as witness, projected to the original variables. On the official 2026 main-track benchmark set, `hydra_satsuma` exceeds the bare winner by a small margin — 335 vs. 334 solved, and PAR-2 2026 vs. 2062 under competition certification scoring — with the whole margin traceable to the structural stages, whose cost elsewhere is below measurement. The factoring stage turns the set's two satisfiable multiplier instances from 30 and 41 s into 0.08 and 0.21 s at a measured cost of at most 0.45 s on any other instance (7 s over the whole run). The circuit stage, run over the whole set as `hydra_circuit_satsuma` under the same protocol, refutes three tree multipliers that no arm had reached — gm16spwtrc in 1,796 s, gm20spctrc in 2,307 s and gm16spwtcl in 4,957 s, the last at the edge of the budget, every certificate checked after the verdict — and loses nothing: 338 solved, 197 certified, PAR-2 1981.

1 Pipeline

The backend runs the following stages in order; the first stage to reach a verdict ends the run. Stage budgets and caps are given with each stage.

1.1 Cook-shape prover

A syntactic detector (inputs up to 10^6 clauses) recognizes five structured UNSAT families, reading the exact clause layout rather than solving:

- **PHP-n-m** — the canonical pigeonhole formula, n pigeons, m holes, $n > m$;
- **RoundRobin** — sports-scheduling infeasibility, a per-pair/per-day two-level pigeonhole;
- **Embedded pigeonhole** — P disjoint at-least-one clauses over S complete slot-mutex cliques (a *slot* generalizes a hole: a hole for PHP, a day for RoundRobin) with $P > S$, the generalization covering e.g. MVRoundRobin;
- **Clique-coloring** — a graph asserted to contain a K-clique yet be C-colorable with $K > C$, proved by composing the clique-membership and coloring layers;
- **Mutilated chessboard** — bipartite perfect-matching infeasibility with imbalanced sides, proved by the two-sided cardinality argument.

On a match the verdict is UNSAT *by construction*, in microseconds to milliseconds, and — when a proof is requested — the module emits the corresponding pseudo-Boolean refutation in VeriPB format. For PHP the proof is exactly Cook's 1976 construction: fresh variables $Q_{ij} = P_{ij} \vee (P_{im} \wedge P_{nj})$ encode a pigeonhole instance with one fewer pigeon and hole; the reduction iterates down to a trivially refutable base. Each layer is polynomial, rendered as VeriPB `red` steps with the definition as witness. This matters because Haken's 1985 lower bound makes PHP exponential for resolution: a solver-emitted DRAT proof (which cannot introduce variables the solver never used) is out of reach at scale, while the extension-variable route is short, and VeriPB is one of the three checkers on the SAT Competition 2026 menu (§2.2). Every detector verifies the clauses its proof consumes, not only the at-least-one layers: the clique-coloring detector once accepted the two positive layers alone, and the unit-propagated residual of a *satisfiable* 6×6 multiplier matched its arity profile (found 2026-09-15 while building the factoring stage; no competition instance was affected). It now looks up every mutex its rup steps use, directly or through the standard encoding's edge variables.

1.2 Factoring stage

For inputs up to 5×10^6 clauses, the stage tests for a multiplier circuit $out = a \cdot b$ whose output bits are pinned by unit clauses to a number N with the factor bits free — the shape of the ezfact and pyhala-braun competition families. The structure is read from the clauses and the encoding conventions from the circuit's own behaviour:

- **Structure.** Parity relations (a 3- or 4-variable set carrying every clause of one parity) are the sum cells; union-find over their unpinned members gives the product columns. The factor bits are the unpinned, relation-free variables that meet at least four others in gate clauses (the 4-core of the co-occurrence graph: a factor bit meets every bit of the other factor in its partial-product gates, a gate output only its few inputs); a bipartite split separates the factors and the addition table (which pairs meet in which column) labels the bits by weight. Each bit's polarity is read off its AND gates, because a shuffled instance flips every variable independently.
- **Conventions.** 48 random factor assignments are propagated through the pin-free CNF. Pins the factor bits never determine and that never conflict are the circuit's constants; pins that never vary are side constraints (pyhala-braun pins "the factor is not 1"); every other pin is matched against the bits of the product under the readings of the factor bits — orientation of the labelling and an implicit odd low bit per factor — and a reading that explains the pins fixes N. The pyhala-braun labelling comes out mirrored; the orientation reading absorbs it.

- **Numbers.** Pollard–Brent with Miller–Rabin on N; every divisor pair of the circuit's widths is tried by propagation through the *full* CNF, and the resulting model is checked against every clause — a wrong reading can only miss, never mislead.

Verdicts: a fitting pair gives SAT with the model as witness (self-certifying, like the pure-XOR path). A number with no fitting pair is a *numeric* UNSAT — reported in the log (`factoring-verdict=unsat`) but never as the verdict, because the primality of N has no short certificate in DRAT, LRAT, SR or VeriPB; the instance proceeds to the later stages and the engine's refutation is the certificate. Anything unrecognized falls through at the structure pass. On the benchmark families the stage reads and solves ezfact16/32/64 (N the square of a 16- or 32-bit prime, or a 16-bit prime) and pyhala-braun 30/35/40 (satisfiable and unsatisfiable, $N = 23173 \times 23197$, 131101×131113, 741457×741469); toughsat is not recognized — its product bits are folded into the last cells as constants that unit propagation reads backwards, and the `toughsat_`n`bits` encoding is not an array multiplier at all.

The stage is bounded on every input. Before anything superlinear it checks plausibility in linear time — at least eight pins, at most 4096 candidate factor bits, and parity-relation and pin counts consistent with a multiplier of that many bits (a verification circuit with thousands of units or hundreds of thousands of candidates is rejected here) — and a wall-clock budget of 1 s (`FACTORING_BUDGET_MS`) backstops the rest; a multiplier of competition size is read in well under 200 ms (§5.2).

1.3 XOR/GF(2) stage

For inputs up to 5×10^6 clauses, the stage recovers XOR constraints from their CNF encodings (complete canonical encodings only; the certifier re-derives the recovery strictly, so no trusted parsing) and runs GF(2) Gaussian elimination. Three outcomes:

- **Inconsistent system** (by unit propagation or by GE): the formula is UNSAT. When a proof is requested the stage emits a parity refutation following Gocht and Nordström's method [4] (henceforth **GN21**): parity reasoning, though exponential for resolution, has short pseudo-Boolean certificates, because cutting planes' *division* rule captures mod-2 arithmetic — encode each recovered XOR as pseudo-Boolean constraints, and summing two of them followed by division by two derives their GF(2) sum in a constant number of proof steps. Our certificate names the (typically small) refuting subset of recovered XORs and derives the contradiction $0 \geq 1$ from their sum, checkable by `veripb` in milliseconds. (The certifier re-derives the XOR recovery itself from the clausal encodings, so the proof does not trust the recovery code.)
- **Pure-XOR satisfiable**: GE solves the system outright; the model is the certificate.
- **Partial simplification**: GE forces some variables and simplifies the clause set, yielding an equisatisfiable *residual*. What happens next depends on *certified mode* (§1.4).

1.4 Certified mode

The `--proof` flag is the certified-mode switch for the whole hydra family. The subtlety is the partial-simplification case: an engine refutation of the *residual* does not certify the *original* formula — the GE step connecting them is real reasoning that the downstream proof does not contain. Therefore:

- **Certified mode** (`--proof` present; the benchmark harness always passes it): the GE simplification is *discarded* and the engine solves the original formula. Full end-to-end certification beats the simplification — a trade measured to be nearly free (§3).
- **Uncertified mode** (no `--proof`): the engine solves the residual. Verdicts remain sound (GE derives only logical consequences; SAT models are reconstructed by overwriting GE-forced variables), but a residual-path UNSAT is reported explicitly as uncertified for the original.

Emitting a proof *of the GE step itself* (deriving the forced units and simplified clauses via extension variables, in DSR or VeriPB) would close this gap and is well-understood machinery; it is deliberately deferred because measurement shows nothing is currently lost (§3).

1.5 Circuit stage

The `hydra_circuit_satsuma` variant adds a fourth stage between the structural stages and the fall-through, from the library module `logic::circuit` (also the `cnf2aig` command line). It is a certified preprocessor for one encoding family, the GenMul multiplier-verification instances, and it was found by measurement: on gm16spwtrc — a 320,000-gate decision diagram of multiplexers, each selected by a primary input — ABC's `dc2` rewriting [12, 11] in front of CaDiCaL turned a 5000 s timeout into an 1,872 s refutation, and what the rewriting did was recover the partial products, which the CNF spells as two nested multiplexers on the inputs per product; SAT sweeping, the usual circuit tool [9, 10], merged 2% of the gates and changed nothing. The stage reproduces that effect natively, with a proof:

- **Does it apply?** The gates are read off the clauses (AND/OR of any width by pattern, XOR groups, any function of up to three inputs by truth table), oriented and ordered with each cycle of definitions broken at one gate, and the select factoring below is tried without writing anything. The stage applies when factoring finds at least 64 products of inputs, put on at least 5% of the gates and shared by at least 20 gates each on average; a hash or an equivalence miter has products without sharing and is left alone, and inputs over 10^7 clauses skip the stage. On the official set it fires on 13 instances, and its criteria would accept a fourteenth that is over the cap (§5.3). The trial, timed directly on every official instance: median 0.3 s, 90th percentile 4.5 s, maximum 43 s, 569 s over the 400 benchmarks.
- **Probe.** CaDiCaL on the original for 20,000 conflicts, logging its proof. Array multipliers fall here in a few hundred conflicts, and the passes would cost them seconds to minutes; a probe verdict is a verdict, CaDiCaL's DRAT its certificate.
- **Select factoring.** Where a gate selects between an inner multiplexer and that multiplexer's own else-branch, $o = s\,?\,(s_2\,?\,a : e) : e$ becomes $o = (s \wedge s_2)\,?\,a : e$. The product $s \wedge s_2$ is one extension variable shared by every gate that tests the same pair, defined by three clauses that are RAT on the new literal; the rewritten gate's clauses are derived from the old ones by unit propagation or by cases on the selects, and a gate that tests the same variable twice folds. Gates nothing reads any more are dropped, their definitions kept aside for model extension. On gm16spwtrc: 140,523 gates rewritten onto 265 products in 6 s, 1.92 M clauses to 1.08 M.
- **Cut-based re-encoding.** The rewritten circuit is written again from cuts of up to six leaves chosen by area flow over four rounds: each cut's function, a 64-row truth table evaluated from the gates between root and leaves, is covered by its irredundant prime-implicant covers (Minato–Morreale [14, 15]) in both polarities, and each resulting clause is derived by cases from those gates' definitions; a clause identical to an original one is kept rather than re-derived. This is the lean cover ABC's CNF writer gives the solver [13] — a multiplexer is four clauses, not six, and a cover spans the gates inside a cut — and it is where most of the gain lives: factoring alone refutes gm16spwtrc in 3,071 s, factoring then cuts in 1,423 s.
- **Solve and certify.** CaDiCaL (the vendored 3.0.1) takes what the passes leave, with the rest of the budget and its own proof. Each pass emits a DRAT prefix deriving its output clauses from its input and deleting what it replaced; the prefixes concatenate, the solver's proof appends, and `drat-trim` checks the one file against the formula the stage was given — after the verdict, in a second bounded phase (`--circuit-verify-secs`, default the solve timeout), exactly as the SR chain's. A SAT model is extended to the original variables through the dropped definitions by a CaDiCaL call that fixes only the variables of the final formula, and is checked against every original clause before it is printed.
- **Certified mode.** The stage is handed the original formula, never the GE residual of §1.4, so its certificate covers the input.

What the stage is *not*: a general circuit preprocessor. Certified SAT sweeping — the fraiging of [10]: simulation-refined candidate classes, whole cones in a persistent CaDiCaL, the solver's lemmas as the proof — is in the same module and was built first; on the unsolved circuits of the set (gus-md5, slp-aes, sncf, sum_of_3_cubes) neither sweeping nor ABC's rewriting changes a 5000 s verdict, and on the multipliers sweeping adds nothing to factoring. The stage is a hard floor under one family, with a certificate, and the trigger keeps it off everything else.

1.6 Fall-through: satsuma-iter+kissat

When no structural stage decides the instance, `hydra_satsuma` hands it to the SAT Competition 2026 main-track winner, built unmodified from the official competition tarball (Anders et al.) and run in an Ubuntu 24.04 container:

1. `satsuma fix <cnf> --bsr --proof-file proof.out --out-file sb.cnf` — the Simplify–Order–Break–Repeat algorithm [16] (Best Paper, SAT 2026): iterate *simplify* (CNF preprocessing), *order* (clique-guided variable ordering via the CLIQUER maximum-clique solver on the variable–clause graph), and *break* (adding *only unit and binary* symmetry-breaking clauses, generated systematically along a Schreier–Sims pointwise-stabilizer chain), until only trivial symmetries remain. Because breaking adds only units and binaries, simplification can shrink the formula — often below the input size — and expose new symmetries for the next round. The emitted *binary SR proof* justifies each breaking clause with its symmetry as substitution witness;
2. `kissat sb.cnf proof.out` — the CDCL engine solves the broken formula and *appends* its clausal refutation to the same proof file;
3. on UNSAT, `dsr-trim -f <cnf> proof.out` verifies the *composed* proof — satsuma's SR prefix plus kissat's refutation — against the exact CNF handed over.

The composition is the point: symmetry-breaking clauses are not consequences of the formula (they are satisfiability-preserving, not equivalence-preserving), so a bare engine refutation of the broken formula certifies nothing about the input. SR's substitution witnesses make the breaking steps checkable, and because kissat's clause additions are valid SR steps, one checker validates the whole file. SAT models over satsuma's augmented variable space are projected back to the original variables (sound: breaking clauses only remove models).

Operationally the container runs are bounded end to end:

- **Solve phase**: in-container `timeout` at the remaining budget minus 2 s, so the container always self-terminates and is never orphaned by the host-side watchdog. The verdict and solve time are recorded *before* any verification.
- **Verify phase**: a second bounded container run (`--satsuma-verify-secs`, default equal to the solve timeout, matching the harness's proof-check budget convention for the LRAT chain; 0 = unlimited). A verification timeout downgrades the row to sound-but-unchecked; the verdict is never lost.
- **Memory**: optional per-container hard cap (`--satsuma-mem-gb`); an instance exceeding it fails alone.
- **Size guard**: instances with $\geq 10^6$ variables or $\geq 2.5 \times 10^7$ clauses skip satsuma (its literal graph on such inputs is several GB) and run kissat directly on the raw formula; an UNSAT proof remains dsr-trim-checkable, since kissat's steps are the suffix of every composed proof.

The sibling backends complete the family: `hydra` (same structural stages, CaDiCaL fall-through with native LRAT checked by `cake_lpr`) and `satsuma` (the winner bare, no structural stages — the baseline arm for measuring what the dispatch adds).

2 Certificates by path

Path	Verdict	Certificate	Checker
Cook shape	UNSAT	Cook-1976 extension proof (VeriPB)	`veripb`
XOR inconsistent	UNSAT	GN21 parity proof (VeriPB)	`veripb`
Pure-XOR solvable	SAT	model witness	—
Multiplier factored	SAT	model witness (propagated from the pair)	—
Multiplier, no factor pair	(none: annotated)	the engine's, below	
Circuit stage, probe	UNSAT	CaDiCaL DRAT	`drat-trim`
Circuit stage, passes	UNSAT	DRAT prefixes + CaDiCaL DRAT	`drat-trim`
Circuit stage	SAT	model, extended to the input	—
satsuma+kissat	UNSAT	composed binary SR	`dsr-trim`
satsuma+kissat	SAT	model (projected)	—
GE residual (uncert. mode)	UNSAT	none for the original	—

Verification of the SR path happens *in-solver* (the second container phase), and the harness credits it from the solver's `dsr-trim VERIFIED UNSAT` marker rather than re-checking — binary SR is not readable by the LRAT/GRAT/VeriPB checkers, and routing it to one of them would produce a spurious rejection. The circuit stage's `drat-trim` check is credited the same way, from its `drat-trim VERIFIED UNSAT` marker.

2.1 Checker cost: hinted vs. unhinted

The two verification chains do structurally different work. `cake_lpr` consumes LRAT, where every step carries unit-propagation hints computed by CaDiCaL at solve time; checking is a near-linear replay. `dsr-trim` consumes unhinted DSR — like `drat-trim`, it must find every propagation itself and discharge each SR step's redundancy goals, i.e. it performs the elaboration that the LRAT path amortized into the solve. A same-CNF measurement on the paper's single test platform — an Apple M4 Pro Mac mini (10 performance + 4 efficiency cores), 64 GB RAM, macOS 26, the satsuma toolchain in an Ubuntu 24.04 Docker VM; every run in this note uses it — (instance `b20`, ISCAS circuit, UNSAT):

Chain	Proof	Size	Check time
CaDiCaL native LRAT	hinted	40 MB	`cake_lpr` 1.55 s
satsuma+kissat DSR	unhinted	20 MB	`dsr-trim` 7.8 s

SR's compensation is succinctness (the same instance's SR proof is half the LRAT's size; Codel–Avigad–Heule report SR proofs at 0.4% of the line count of their RAT translations) and expressiveness (the symmetry steps have no DRAT analogue without extension variables). Memory behaves oppositely: `dsr-trim` is a compact C program (no out-of-memory events across concurrent checking in our runs), while `cake_lpr`'s CakeML runtime requires a pre-sized heap that scales with the proof and is the component our harness bounds with a dedicated memory cap and a concurrency semaphore.

2.2 Competition-format compliance

SAT Competition 2026 accepts three proof checkers in the main track: DPR (`dpr-trim`), GRAT (`gratgen`), and VeriPB (`veripb`). The Cook and GN21 certificates are VeriPB proofs and verify under the same `veripb` the competition runs; the SR chain is the 2026 winner's own; the circuit stage's certificates are plain DRAT, the RAT fragment of DPR, which `dpr-trim` reads unchanged. One gap separates `hydra_satsuma` from literal entry shape: the rules have a solver declare a *single* checker and write one `proof.out`, whereas this backend emits per-path formats. Unification is mechanical future work — either translate the engine's clausal refutations into VeriPB (`rup`/`del` steps map directly; RAT additions become `red` with witness) and declare VeriPB for everything, or render the Cook/GN21 constructions in DRAT-with-extension-variables and declare the clausal chain. The VeriPB route is helped by satsuma itself: its SR steps can be emitted in either DSR or VeriPB format [16], so only the engine refutation needs translating.

3 Measured design decisions

Two measurements on the official 2026 main-track set (400 instances with duplicates; the M4 Pro Mac mini) fixed design choices in advance of the full performance run:

- **GE simplification is worth ≈ nothing on this set.** Every instance on which the partial-simplification path fires (twelve, all ISCAS-class circuits) recovers exactly one XOR and forces exactly one variable — on formulas of 2×10^3 to 3.4×10^5 variables. Solve-only A/B (original vs. residual, same engine): `b20` 7.43 s vs. 7.40 s, `s38417` 1.66 s vs. 1.68 s, `c7552` 0.75 s vs. 0.77 s — within noise. Certified mode's "ignore GE, solve the original" therefore costs nothing measurable, and building the GE-step certifier (§1.4) is not currently justified by performance.
- **Where the XOR stage does pay, it is already certified.** In the prior full `hydra` run on this set, eleven instances were decided outright by the parity stage in ∼10 ms each, all eleven with verified GN21 certificates; twenty-seven more fell to the Cook prover with verified VeriPB proofs.

4 The benchmark set: construction and comparability

4.1 Construction

The evaluation set is the official SAT Competition 2026 main-track selection, reconstructed as follows and auditable at every step:

- **Source of truth**: the competition's benchmark-compilation-script tarball (https://satcompetition.github.io/2026/downloads/) carries the authoritative `selected_benchmarks.csv` with columns `(isohash2, hash, author, family, result)`. The file is archived in-repo at `tools/gbd/sc2026_selected_benchmarks.csv`.
- **Slot count and duplicates**: the CSV has exactly **400 rows over 391 distinct hashes**: nine hashes each appear twice, *in the official selection itself* (the ISCAS circuit `b20`, hash `c4318c6a…`, is one of them). Re-verified from the archived CSV and from the built index at the time of writing.
- **Index**: `main_track_2026_official.jsonl` is built row-for-row from the CSV (400 records), each carrying the hash, the original filename, the family/author metadata, and the competition's expected result where known.
- **Instance identity**: instance files are hash-addressed (`<hash>-<name>.cnf.xz`) and were fetched from the GBD benchmark database *by* those hashes; the GBD hash is the MD5 of the normalized formula (not of the file bytes), so agreement between the official CSV and the on-disk files is content-level identity under GBD normalization, not filename trust.
- **Dedup semantics**: the harness by default deduplicates to the 391 unique instances and prints `deduped 9 duplicate records (kept 391 unique)`; `--no-dedupe` runs all 400 benchmarks exactly as officially scored (a duplicated instance counts twice).
- **Result integrity**: every row is cross-checked against the competition's expected status where recorded (any SAT/UNSAT disagreement is flagged as a mismatch; none has been observed), SAT verdicts carry validated witnesses, and UNSAT verdicts carry the per-chain certificates of §2.

4.2 Comparability with the official 2026 standings

Absolute solved counts measured here must *not* be read against the official competition standings. The official main-track winner, satsuma-iter+kissat, scored **276/400** (PAR-2 3647.02) under 5000 s CPU per instance, one instance per node, on the competition's cluster hardware. Our protocol runs 5000 s *wall-clock* per instance with *ten instances concurrently* on one 14-core Apple Silicon machine (64 GB host; the satsuma-based arms execute inside a Linux VM: 47 GB for the August runs, 32 GB with memory-aware scheduling for the October one, §5.3). Per-core, per-second compute on the M4 Pro substantially exceeds the competition nodes'. For reference:

Single-thread benchmark	Apple M4 Pro	Xeon Platinum 8368
PassMark ST	4506	2345

The local side is the Apple M4 Pro Mac mini described in §2.1. SAT Competition 2026 ran — for the first time — on the NHR@KIT system *HoreKa*, Blue partition, whose documented nodes are dual Intel Xeon Platinum 8368 (Ice Lake, 2021; 76 cores and $\geq$ 256 GB per node). (An earlier draft assumed the LMU/SV-COMP cluster from the BenchExec-style Dockerfile in the winning submission's archive; that Dockerfile pins the benchmarking *tooling*, not the site.) With 76-core nodes and per-run CPU-time and memory limits (30 GB per solver in 2025), many solver runs share a node — so *both* environments schedule concurrent runs against shared memory, ours ten per 14-core machine, theirs batched per node, and neither side's times are contention-free. Published single-thread figures put the M4 Pro P-core at roughly 1.9× the 8368 core (PassMark 4506/2345); CPU-second vs. wall-second accounting differs on top. Equal nominal seconds still buy substantially more search here, which on heavy-tailed runtimes pulls many near-timeout instances under the line — and is why local solve counts (e.g. the `hydra` baseline's 311 of the 391 unique instances, 318 of the 400 official benchmarks) run well above the official 276/400. The bare `satsuma` arm turns this from estimate into measurement: it is the winner's own pipeline, built unmodified from the official submission archive, on the official instance set — so its local 334/400 against the official 276/400 is a pure environment delta. Reading it off the cactus curve: local `satsuma` reaches 276 solves at a per-instance budget of only **653 s** in its August run and 691 s in its October rerun, where the competition needed its full 5000 s (CPU) for the same count — an effective environment factor of $5000/653 \approx \mathbf{7.7}\times$, or 7.2× in October. This is the all-in environment difference — per-core hardware, node scheduling and contention on both sides, and CPU- vs. wall-second accounting fold into it, and count data alone cannot decompose them. (The estimate is also count-based, not instance-matched: official per-instance results are not published alongside the standings.) Notably, the single-thread microbenchmark ratio above ($\approx 1.9\times$) *dramatically* underpredicts the measured end-to-end factor: CDCL search is memory-latency-bound and scheduling-sensitive, and neither effect is priced by an arithmetic microbenchmark — a reminder that microbenchmark constants do not price real workloads. Local solve counts therefore overshoot official standings mechanically, by roughly this factor's worth of extra effective budget per instance. Certification in §5 is scored under the competition's own conditions — the 45,000 s-per-proof checking budget (9× the solve timeout; stated for SC2020 and SC2025, the standing convention) with ample checker memory — so no checking-budget asymmetry remains. The comparisons this note will report (§5) are therefore *internal*: all four arms under one fixed protocol on one machine, where hardware and scheduling effects cancel and only the solver pipelines differ. The official figures are quoted for context only.

5 Results

We report these results *as if the competition were run on an Apple M4 Pro Mac mini* (§2.1), with the satsuma-toolchain components (satsuma, kissat, `dsr-trim`) executing under Ubuntu 24.04 in Docker containers. All four arms completed the official 400-benchmark set at 5000 s per instance in certified mode, running ten benchmark processes in parallel (`-j 10`) so as to fully utilize the machine's ten performance cores. Certification is scored under the competition's conditions: an UNSAT instance counts toward PAR-2 only if its proof verifies within the official 45,000 s checking budget, with checker memory ample for the chain (§5.4); a check that exceeds the budget leaves the instance *not solved* for scoring, per the official rule. Soundness was clean across all four runs: zero expected-status mismatches, zero witness failures, zero contradictory verdicts between any pair of arms, and no proof *rejected* by any checker anywhere in the study.

Arm	Solved/400	SAT	UNSAT	UNSAT certified	PAR-2*
`hydra` (August)	318	132	186	185	2483
`satsuma`	334	139	195	193	2062
`hydra_satsuma`	335	139	196	194	2026
`hydra_circuit_satsuma`	**338**	139	199	**197**	**1981**

*competition scoring: SAT instances and *certified* UNSAT instances contribute their solve time; uncertified UNSATs and timeouts contribute 2×5000 s. The `hydra` row is the August 2026 run; the other three are the October reruns, scored by the

same rule with the official-budget re-check of §5.4 carried over instance by instance (satsuma and kissat are deterministic, so a rerun writes the same proof; checked on one instance), by a script that reproduces the August figures to within 0.2 PAR-2 points. The August runs gave `satsuma` 334 / 139 / 195 / 193 / 2064 and `hydra_satsuma` 335 / 139 / 196 / 194 / 2037 (2037.17 with the factoring stage, which the October run includes); a first run of `hydra_circuit_satsuma` on 2026-09-30, before the memory-aware scheduling below, gave 337 / 139 / 198 / 196 / 2002, gm16spwtcl 2 s over the budget.

All four runs use the identical protocol on all 400 official benchmarks (the selection's nine duplicate hashes each appear as two benchmarks and count twice, as in official scoring). `hydra` ran in August 2026; the three satsuma-based arms ran back to back in October, `hydra_circuit_satsuma` on the 1st and 2nd, `hydra_satsuma` on the 2nd and 3rd, `satsuma` on the 3rd and 4th.

Conditions. The satsuma-based arms were rerun because their first runs did not share conditions. Two things had gone wrong with memory, both found while running `hydra_circuit_satsuma` on 2026-09-30, which came out 5% slower per instance than the August `hydra_satsuma` run. The Docker VM's guest caches the proof files kissat streams through the bind mount, so the VM grows to its 47 GB cap and the host compresses and swaps it; and the twelve md5 instances, up to 315M clauses, all start in the first minute, kissat taking 20 GB on the largest while the host's own `sat` process holds 14 GB of parsed formula. The fix was measured into two changes: the harness now admits instances by their estimated footprint, sized from the file without decompressing it, so the big ones run two or three at a time rather than ten, and the VM is capped at 32 GB. Under those conditions the host compressor averaged 4 GB across each October run against 38 GB on 2026-09-30. Per instance, on the rows above 5 s solved in both, `hydra_satsuma`'s rerun is 4% faster than its August run (geometric mean 0.960; the md5 family 17%) and `satsuma`'s 1% (0.992), and the two October runs are within 1% of each other, where the August pair differed by 2–3%. The spread that remains is the machine, not the search: satsuma and kissat are deterministic on identical input (checked: identical output and proofs twice), two identical kissat runs beside ten solvers differed by 9%, and macOS puts a runnable thread on one of the four efficiency cores whenever more than ten are runnable, as they were throughout. Rows within 10% of either 5000 s cliff should be read as ties. One effect of running back to back: the VM keeps the memory it has grown into, so each run after the first began on a fuller host, which shows in its md5 block, the first ten minutes.

Engine effect (`hydra` vs. `hydra_satsuma`, structure held fixed): replacing CaDiCaL by satsuma-iter+kissat is worth +17 verdicts and, under certification scoring, −18% PAR-2 (2483 → 2026; the same +17 and −18% against August's `hydra_satsuma`). On the 391 unique instances `hydra_satsuma` solves eighteen that `hydra` does not — concentrated in symmetric design families (both `SC25_Timetable` instances, the `lockchart` families, `ramsey_4_4_18`, two `oddball`, a `le450` coloring, a multiplier-equivalence UNSAT) — against three in the other direction with no structural signature; the sets are the same in both months.

5.1 What the structural front-end is worth

The `satsuma` vs. `hydra_satsuma` pair isolates the structural stages exactly: same engine, same container, same protocol, and in October the same conditions, run on consecutive days; the only difference is the structural front-end. Across the 360 benchmarks where no structure applies the two arms' solved sets are *identical* (zero flips in either direction); every count and score difference between them traces to the 40 structural benchmarks plus measurement noise.

Benefit where it applies (40 benchmarks: 27 Cook, 11 GN21, 2 factoring): `hydra_satsuma` decides all 40 in **1.2 s total**, every UNSAT verdict carrying a VeriPB certificate that checks in milliseconds and both SAT verdicts their model. Bare `satsuma` solves 39 of 40 at a PAR-2 cost of 14,489 s: `mchess_20` times out (the run's only count flip), `mchess_17` barely lands at 4,379 s — 88% of budget, and its SR proof then needs a further 28,505 s of checking — `tseitingrid6x185_shuffled`, a Tseitin parity contradiction, takes 14.1 s against the GN21 path's 0.026 s, and the two factoring instances take 30.0 and 40.9 s against 0.08 and 0.21 s. The honest fine print: the other 35 are easy for the winner too (median 0.4 s), so *nearly the whole structural credit concentrates in the mutilated-chessboard pair* — the one family in this set that is resolution-exponential, not rescued by SOBR-style unit/binary symmetry breaking, and trivial for the two-sided cardinality argument. The front-end's value against a symmetry-capable engine is thus a *hard floor under structured families*, plus instant polynomial certificates, rather than a broad speedup: on a set without such families it would contribute

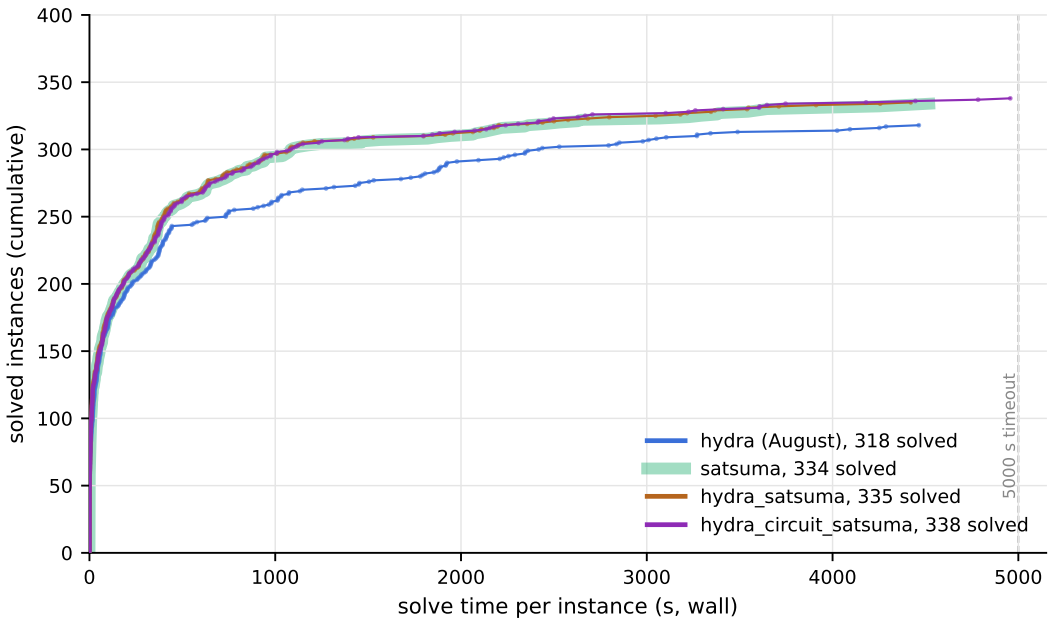


Figure 1: Cactus plot on the official 400-benchmark set: cumulative instances solved within a given per-instance time budget; one dot per solved instance, every arm aligned to the official benchmarks per hash; `hydra` from August, the others from October. `satsuma` is drawn as a wide translucent band beneath `hydra_satsuma`'s thin line because the two curves coincide almost everywhere — that near-total overlap is itself the finding (§5.1): with a symmetry-capable engine, the structural front-end changes the curve only at the chessboard pair, while the engine swap separates both from `hydra` from ∼250 instances on. `hydra_circuit_satsuma` runs along `hydra_satsuma` and leaves it only at its three multiplier refutations, 1,796, 2,307 and 4,957 s (§5.3); it includes the factoring stage, whose two dots sit under 0.2 s.

certification and latency only; on a set with more of them (larger chessboards, larger Tseitin grids — both grow without bound) the count gap widens mechanically.

Overhead where it does not apply (360 benchmarks): measured as the median paired solve-time difference on the 295 benchmarks both arms solved — same formula, same engine, same conditions, so the pairing isolates the detection prefix — the cost is +0.01 **s per instance**, and the paired differences sum to +20 s over the whole run: nothing measurable. The prefix's direct costs agree: milliseconds for the Cook and XOR detectors, 7 s over the run for the factoring stage (§5.2). The August pair gave +1.16 s per instance and a sum of +3,684 s, which the October runs show was the two runs' conditions rather than the prefix. Detection never caused a flip: no instance was lost to overhead.

5.2 The factoring stage on the official set

The official set has four prime-factoring benchmarks. Re-run through the same harness (official index, 5000 s, `hydra_satsuma` with the stage) on 2026-09-16, against the August `hydra_satsuma` run, which did not have it:

Benchmark	August run	with the stage	
ezfact64_6	SAT 30.0 s	SAT 0.07 s	factored; witness re-evaluated on all 19,785 clauses
pyhala-braun-sat-40-4-03	SAT 42.5 s	SAT 0.20 s	factored; witness re-evaluated on all 31,795 clauses
pyhala-braun-unsat-40-4-02	UNSAT 43.2 s	UNSAT 36.9 s	numeric UNSAT annotated in 0.2 s; the certificate is kissat's SR proof, `dsr-trim` verified
toughsat_factoring_895s	SAT 279.9 s	SAT 225.8 s	not recognized; engine path unchanged

Only the two satisfiable multiplier instances change hands: they are decided by the stage in milliseconds, −72.3 s over the run, −0.18 PAR-2 points (2037.35 to 2037.17). The October runs include the stage from the start: 0.08 and 0.21 s in `hydra_satsuma` against 30.0 and 40.9 s in `satsuma`. The unsatisfiable one gains nothing real — the stage knows the answer in 0.2 s, but the certificate is still the engine's refutation, so its time is the same within run-to-run variation, as is toughsat's. Solved counts and certified counts are unchanged.

Cost elsewhere. The stage is a structure pass on every other instance, and its cost was measured directly on all 391 unique official instances rather than inferred from paired runs. The 57 instances above the 5×10^6-clause cap skip it. On the other 331 (three of them the recognized multipliers) the median is 0.2 ms, the 90th percentile 70 ms and the maximum 0.45 s (`q_query_3_L200_coli`, 3.5×10^6 clauses): 218 are rejected by the pin count before any superlinear work, and the rest pay the parity-relation pass, linear in the formula. The total is 7.0 s over the run — 0.017 PAR-2 points, below the resolution of the paired-difference measurement of §5.1. The bounds of §1.2 earned their place: the first, unbounded version of the stage cost 238 s over the same set, 44 s of it on one verification circuit (`x-epic_a19-p16_step`) whose 427,000 candidate bits sent the candidate filter into a quadratic peel — the measurement that produced the plausibility bounds, the incremental peel and the 1 s budget.

5.3 The circuit stage on the official set

`hydra_circuit_satsuma` ran the official index under the protocol of §5 twice: on 2026-09-30, and again on 2026-10-01/02 after the memory problem described under *Conditions* in §5 was found and fixed; the figures here are the second run's, twenty-one hours for the 391 unique instances, compared with the October `hydra_satsuma` run. The stage's criteria accept 14 of them: the twelve GenMul multiplier-verification instances, `x-epic_a10-p53_transition` and `PRP_40_40`; one, gm32spctlf at 14M clauses, is over the clause cap, so the stage runs on 13. On the other 377 it looks and steps aside, and their verdicts are `hydra_satsuma`'s to the instance — no flip in either direction, `hydra_satsuma`'s 65 timeouts reduced to 62 by exactly the three gains below.

Benchmark	`hydra_satsuma`	`hydra_circuit_satsuma`		certificate
gm16spwtrc	timeout	UNSAT	1,796 s	`drat-trim` 598 s
gm20spctrc	timeout	UNSAT	2,307 s	`drat-trim` 195 s
gm16spwtcl	timeout	UNSAT	4,957 s	`drat-trim` 1,319 s
gm16spctrc	UNSAT 525 s	UNSAT	60 s	`drat-trim` 8 s
gm20sparrc, gm24sparrc, gm28sparrc	UNSAT 0.7–12.4 s	UNSAT	0.6–8.2 s	probe; `drat-trim` <1 s
gm32sparrc, gm36sparrc	UNSAT 5.2–8.4 s	UNSAT	6.4–11.3 s	probe; `drat-trim` <1 s
x-epic_a10-p53_transition	UNSAT 9.7 s	UNSAT	29.5 s	`drat-trim` 172 s
PRP_40_40	SAT 29.0 s	SAT	29.9 s	the model
gm24spctbk, gm24spctrc	timeout	timeout		
gm32spctlf (over the cap)	timeout	timeout		

Three refutations no arm had reached. gm16spwtrc, gm20spctrc and gm16spwtcl are tree multipliers, 16 and 20 bits, where the array multipliers of the family fall to the probe in a few hundred conflicts; neither the winner's pipeline nor `hydra`'s CaDiCaL reaches them in 5000 s here, in August or in October, at the 7 to 8× effective budget of §4.2, and the competition's selection lists their status as unknown. All three certificates — the passes' prefixes followed by CaDiCaL's refutation, one file — were accepted by `drat-trim`

against the original files; under the competition's accounting they are solved instances. The third is a cliff row and should be read as one: gm16spwtcl was refuted at 4,324 s in a run of the 14 alone (`doc/competition/competition-benchmark_circuit14_5000_hydra_circuit_satsuma.md`), timed out by 2 s in the full run of 2026-09-30, when its neighbours' memory traffic slowed the circuit rows by 10 to 15%, and came in at 4,957 s in the run reported here. The firm gain under competition conditions is +2, with a third instance that lands on either side of the budget by a percent.

Where the stage costs. gm16spctrc, the one tree multiplier the engine solves, goes from 525 to 60 s. `x-epic_a10-p53` is the one instance the stage makes slower: the trigger fires, the probe is undecided, and the passes plus CaDiCaL take 30 s against the engine's 9.7 s, with 172 s of checking after. `PRP_40_40` is the stage's one SAT, found by CaDiCaL on the rewritten formula in 30 s against the engine's 29 s, the model extended to the original variables through the dropped definitions and checked against every clause.

Cost where the stage declines. There its trial is the only cost, so it was timed directly on every official instance, four at a time on an otherwise idle machine, from the stage's own clock: 296 instances pay it, median 0.3 s, 90th percentile 4.5 s, maximum 43 s (`baseballcover12with25`, 7M clauses); 42 skip it above the clause cap and 40 are decided before it by hydra's stages. That is 569 s over the 400 benchmarks, 1.4 PAR-2 points. The paired comparison with the October `hydra_satsuma` run sees the trial where it is large — `ncc_none_5047` 26 s slower against a 23 s trial, `x-epic_a19-p16` 18 s slower against 10 s — but is dominated by something else: on the 287 benchmarks the engine decides in both runs, the circuit run is 4% slower per instance, and the excess grows with solve time, a median of 0.4 s under a minute and 25 s above ten minutes. A fixed prefix cannot produce that shape, and the stage does nothing while the engine runs, so it is the runs' conditions, the circuit run going first in the October sequence; it is larger than the 1% between the other two October runs, and these data do not attribute it further. No instance was lost to the trial.

Engines. Of the 338 benchmarks solved: satsuma-iter+kissat 287, the Cook prover 27, GN21 11, the circuit stage 11 (ten UNSAT, one SAT), the factoring stage 2.

Certification. Every UNSAT the stage claimed checked in-run. Eighteen satsuma proofs exceeded the harness's 5000 s check budget, all of them among the nineteen of the October `hydra_satsuma` run and the twenty-five of August's; sixteen of them verified under the official 45,000 s budget in the re-check of §5.4, and the two that did not are the two proofs uncertifiable in every arm. Hence 197 certified of 199.

5.4 Certification economics: the proof chains

The arms emit certificates through four chains with sharply different cost profiles; which chains an arm can use is set by its pipeline. `hydra` certifies structural verdicts with VeriPB proofs and engine verdicts with CaDiCaL's native LRAT; `satsuma` certifies everything with binary SR; `hydra_satsuma` uses VeriPB for its 38 structural verdicts and SR for the 158 engine verdicts; `hydra_circuit_satsuma` adds a fourth chain, plain DRAT checked by `drat-trim`, for the circuit stage's ten.

Chain	Producer	Checker	Check speed	Check memory
VeriPB (Cook/GN21)	structural stages	`veripb`	ms–s	negligible
LRAT (hinted)	CaDiCaL	`cake_lpr`	s–min	**the binding axis**
SR (unhinted)	satsuma+kissat	`dsr-trim`	**the binding axis**	light
DRAT (unhinted)	circuit stage + CaDiCaL	`drat-trim`	s–min	the proof's size

VeriPB (38 certificates in each hydra arm): the structural proofs are polynomial constructions checked in milliseconds to seconds with negligible memory — 76 checks across the two arms, zero at any meaningful cost. When a structural stage fires, its certificate is effectively free.

LRAT + `cake_lpr` (`hydra`'s 148 engine certificates): CaDiCaL computes unit-propagation hints during the solve, so checking is a near-linear replay — 1.55 s for a 40 MB proof (`b20`), 58.6 s for `stone-width3chain`, 62.8 s for the heaviest verified LRAT in the study. Speed is not the constraint; *memory* is: `cake_lpr`'s CakeML heap scales with the proof, and of 148 proofs, two needed more than a 16 GB heap and one (`hash_table_find_safety_size_21`) exceeded even 48 GB. The checker is formally verified — the strongest trust story of the three.

SR + `dsr-trim` (satsuma-chain certificates: 195 in `satsuma`, 158 in `hydra_satsuma`): SR proofs are succinct (half the LRAT's size on the same instance) and can express symmetry reasoning clausal hints cannot, but they carry no hints, so the checker performs the propagation search itself — checking cost rivals or exceeds solving cost. On the heavy tail, median check time was 8,671 s with verified checks up to 28,505 s (`mchess_17`); two proofs exceeded the official 45,000 s budget outright (`stone-width3chain`, `hash_table`). Memory, by contrast, never mattered: `dsr-trim` is a compact C program that ran ten-concurrent without incident throughout.

The chains cross. `stone-width3chain` is the clean head-to-head: a ∼1-minute solve in every arm, certified by `hydra`'s LRAT chain in 58.6 s while its SR proof is uncheckable within the official budget — a >750× gap on one instance, purely from hints existing. `hash_table` is the double defeat: its SR proof is time-unbounded *and* its LRAT proof is memory-unbounded (beyond a 48 GB heap), making it the one instance in the set uncertifiable under competition conditions by either engine chain. And `mchess_17` completes the triangle: SR certifies it only after 28,505 s of checking, LRAT never sees it, and the Cook prover certifies it in a millisecond — on structured instances, the structural chain dominates both engine chains on every axis at once.

Reading. The comparison resolves cleanly. The engine is worth +16/+17 verdicts over CaDiCaL on this set; the structural front-end is worth +1 verdict, +1 certified instance, and 36 PAR-2 points against the strongest available engine — at no measurable cost in detection and zero lost instances; within it, the factoring stage turns two 30–41 s solves into milliseconds, for a prefix of at most 0.45 s (7 s over the run); the circuit stage adds three certified refutations no arm had reached, one of them at the budget's edge, and 44 PAR-2 points, on the one family it is built for, for a trial of at most 43 s on any instance (1.4 PAR-2 points over the run) and no instance lost. Its deeper contributions are invariants the count does not show: instant polynomial certificates on structured families (against engine-time SR proofs that are expensive to check — and on this set twice uncheckable even at the official budget), and a hard floor that is unbounded in the direction the engine's weakness grows.

Acknowledgments

Claude (Fable 5 and Opus 5.5, Anthropic) assisted with the implementation, measurement, and drafting throughout. satsuma and dejavu are by Markus Anders et al.; kissat by Armin Biere; `dsr-trim` and `lsr-check` by Cayden Codel; `cake_lpr` by Yong Kiam Tan, Marijn Heule, and Magnus Myreen; VeriPB by Stephan Gocht, Jakob Nordström, et al. The satsuma-iter+kissat components are built unmodified from the official SAT Competition 2026 submission archive and run as-is in a container.

References

[1] S. A. Cook. The complexity of theorem-proving procedures. *STOC*, 1971.

[2] S. A. Cook. A short proof of the pigeon hole principle using extended resolution. *SIGACT News*, 8(4):28–32, 1976.

[3] A. Haken. The intractability of resolution. *Theoretical Computer Science*, 39:297–308, 1985.

[4] S. Gocht and J. Nordström. Certifying parity reasoning efficiently using pseudo-Boolean proofs. *AAAI*, 2021.

[5] C. R. Codel, J. Avigad, and M. J. H. Heule. Verified substitution redundancy checking. *FMCAD*, 2024. https://www.cs.cmu.edu/~mheule/publications/SRcheck.pdf

[6] Y. K. Tan, M. J. H. Heule, and M. O. Myreen. `cake_lpr`: Verified propagation redundancy checking in CakeML. *TACAS*, 2021.

[7] F. Pollitt, M. Fleury, and A. Biere. Faster LRAT checking than solving with CaDiCaL. *SAT*, 2023.

[8] SAT Competition 2026: certificates and checkers. https://satcompetition.github.io/2026/output.html; winner submission archive https://satcompetition.github.io/2026/downloads/solvers/anders.tar.xz.

[9] A. Kuehlmann, V. Paruthi, F. Krohm, and M. K. Ganai. Robust Boolean reasoning for equivalence checking and functional property verification. *IEEE Trans. CAD*, 21(12):1377–1394, 2002.

[10] A. Mishchenko, S. Chatterjee, R. Jiang, and R. Brayton. FRAIGs: A unifying representation for logic synthesis and verification. ERL Technical Report, EECS Dept., UC Berkeley, March 2005.

[11] A. Mishchenko, S. Chatterjee, and R. Brayton. DAG-aware AIG rewriting: A fresh look at combinational logic synthesis. *DAC*, 2006.

[12] R. Brayton and A. Mishchenko. ABC: An academic industrial-strength verification tool. *CAV*, LNCS 6174, pp. 24–40, 2010. https://github.com/berkeley-abc/abc

[13] N. Een, A. Mishchenko, and N. Sörensson. Applying logic synthesis for speeding up SAT. *SAT*, LNCS 4501, pp. 272–286, 2007.

[14] E. Morreale. Recursive operators for prime implicant and irredundant normal form determination. *IEEE Trans. Computers*, C-19(6):504–509, 1970.

[15] S. Minato. Fast generation of prime-irredundant covers from binary decision diagrams. *IEICE Trans. Fundamentals*, E76-A(6):967–973, 1993.

[16] M. Anders, C. Codel, and M. J. H. Heule. Simplify, order, break, repeat. *SAT 2026*, LIPIcs vol. 377, article 4, pp. 4:1–4:19. DOI `10.4230/LIPIcs.SAT.2026.4`. Best Paper Award, SAT 2026. Software: DOI `10.5281/zenodo.20185023`; satsuma: https://github.com/markusa4/satsuma.